

Проверка входных параметров методов

Описание проблемы

Зачастую программист не проверяет входящие параметры метода БЛ, полагаясь на то, что метод будет вызван лишь так, как планировалось. Однако ничто не мешает злоумышленнику видоизменять значение и тип параметров, получать в ответ ошибки, раскрывающие информацию об именовании переменных и внутренней кодовой структуре. В некоторых случаях такие ошибки могут стать причиной падения сервиса/сервера.

Пример 1. Передача неопределенных значений:

```
RPC_FUNC_1( L"Test.Echo", String, TestEcho, boost::optional<String>,
L"param", param )
{
    if(!param) // проверяем, что optional не none, иначе получим
неопределенное поведение
        return;
    result = *param
}
```

```
def test_echo(param: Optional[str] = None) -> Optional[str]:

    if param is None:
        return
    result = param
    return result
```

абсолютно аналогично следует работать с unique_ptr:

```
RPC_FUNC_1( L"Test.Echo", String, TestEcho, std::unique_ptr<String>,
L"param", param )
{
    if(!param) // проверяем, что unique_ptr не none, иначе получим
неопределенное поведение
        return;
    result = *param
}
```

Если мы в API декларируем, что в качестве аргумента НЕЛЬЗЯ передавать null, то объявление функции следует сделать так:

```
// при попытке передать null в качестве аргумента платформа сгенерирует
соответствующее исключение
RPC_FUNC_1( L"Test.Echo", String, TestEcho, String, L"param", param )
{
    result = param;
}

def test_echo_(param: str) -> str:
    result = param
    return result
```

Отдельно следует отметить корректные проверки при работе с Record/RecordSet:

```

RPC_FUNC_1( L"Test.Echo", String, TestEcho, Record, L"param", param )
{
    // проверяем наличие поля, к которому хотим обратиться, иначе прилетит
    исключение от платформы
    if( !param.TestField( L"fldname" ) )
        return;
    // проверяем, что поле, к которому хотим обратиться, не null, иначе
    прилетит исключение от платформы
    if( param[ L"fldname" ].IsNull() )
        return;
    // тут сюрпризов уже быть не должно
    result = param[ L"fldname" ].To< String >();
}

def test_echo(param: Dict[str, Optional[str]]) -> Optional[str]:
    if "fldname" not in param:
        return
    if param["fldname"] is None:
        return
    result = param["fldname"]
    return result

```

невыполнение этих проверок не приводит к segfault, однако могут появляться ошибки из платформы, источник которых может быть неочевиден.

Пример 2. Проверка целочисленных значений:

```

void Foo( int param ) // если окажется, что param, например, равен -1, то
получим 4 миллиарда итераций цикла
{
    for( unsigned int i = 0; i < param; ++i )
    {
        // do something
    }
}

```

Ситуация может быть хуже, если аргумент используется для индексации элемента массива:

```

std::vector<int> some_vector;
some_vector.resize( 10 );
int Foo( size_t param )
{
    return some_vector[ param ]; // тут будет UB, если значение param выйдет
за границы размера массива

```

Проверять следует не только параметры, переданные пользователем, но и результаты промежуточных вычислений. Например, после проведения выборки следует проверить наличие в ней записей.

В отличие от C++, в Python такие действия вызывают четко определенные исключения. Однако эти исключения, если их не обработать должным образом, могут привести к отказу в обслуживании и раскрытию внутренней структуры кода.

```

def get_id(product_id_input):
    try:
        product_id = int(product_id_input)
        if product_id < 0:
            return None
        return PRODUCTS[product_id]
    except (ValueError, TypeError, IndexError):
        return None

```

Последствия эксплуатации

- Получение информации о коде.
- Ошибки в логах.
- Отказ в обслуживании, если рабочие процессы будут падать чаще, чем подниматься (у сервиса не будет работоспособных рабочих процессов для выполнения запросов).

Как исправить?

1. Проверять пользовательский ввод
2. Корректно обрабатывать исключительные ситуации